



Coordinación de
Educación Abierta y a Distancia
VICERRECTORADO ACADÉMICO



ASIGNATURA MACHINE LEARNING

Actividad Autónoma 4

Nombre de la actividad: Árboles de decisión y métricas avanzadas

Unidad 2: Modelos avanzados de clasificación y regresión

Tema 2: Árboles de decisión y métricas avanzadas



FACULTAD DE
Ingeniería

Nombre: Lenin Lopez

Fecha: 24/05/2026

Carrera: Ciencia de Datos e IA

Periodo académico: 2026-1S

Semestre: 4to.

Contenido

1. Árboles de decisión: criterios de partición

- Ejecute el siguiente código:

```
# Importa DecisionTreeClassifier para crear el modelo de árbol de decisión y
plot_tree para visualizarlo
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.datasets import load_iris # Importa la función para cargar el
dataset Iris
import matplotlib.pyplot as plt # Importa matplotlib para la visualización de
gráficos

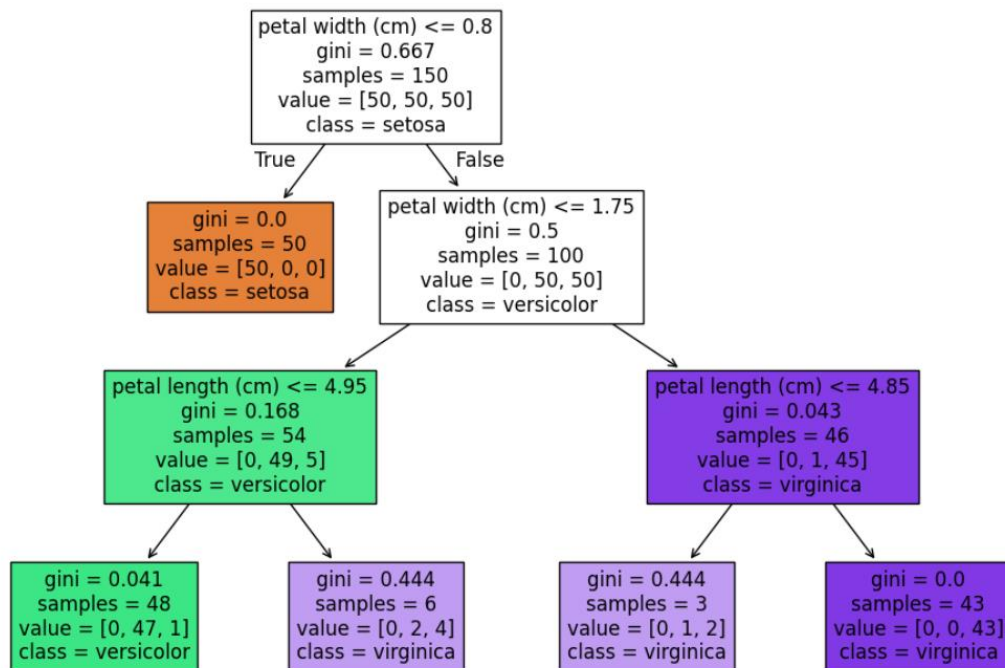
# Cargar el dataset Iris
iris = load_iris()

# Separa las características (X) y las etiquetas (y) del dataset
# X contiene las características (longitud y ancho de sépalo y pétalo) e
# y contiene las etiquetas de clase (0, 1, 2 para las tres especies de iris)
X, y = iris.data, iris.target

# Entrenar árbol de decisión con criterio Gini y profundidad máxima de 3
# y una semilla aleatoria para reproducibilidad
clf = DecisionTreeClassifier(criterion="gini", max_depth=3, random_state=0)
# Entrena el modelo de árbol de decisión con los datos de entrada X y las
etiquetas y
clf.fit(X, y)

# Visualizar el árbol entrenado
# Crea una nueva figura para el gráfico con un tamaño específico
plt.figure(figsize=(12,8))
# Dibuja el árbol de decisión
# Rellena los nodos con colores, usa los nombres de las características y los
nombres de las clases
plot_tree(clf, filled=True, feature_names=iris.feature_names,
class_names=iris.target_names)
# Muestra el gráfico del árbol de decisión
plt.show()
```

- Observar qué variable aparece en la raíz del árbol.

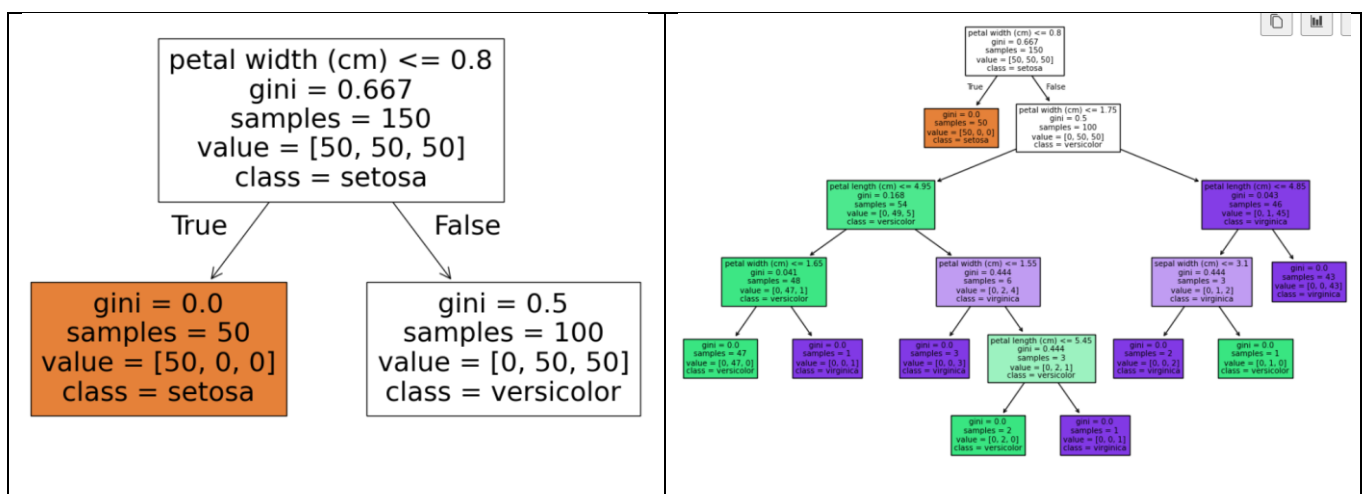


- Explicar por qué se eligió esa variable como la primera división (mayor ganancia de información o menor índice Gini).

En el dataset Iris, el petal width es la característica principal para dividir el dataset con petal width ≤ 0.8 c.m. logra dividir muy eficazmente la especie *Iris Setosa* de las otras dos especies (*Iris Versicolor* y *Iris Virginica*).

- Modificar el parámetro max_depth (ej. 1 y 5) y comparar la diferencia en los árboles obtenidos:

¿Cuál de los dos modelos se ajusta mejor a los datos?



- El modelo que se ajusta mejor al ejercicio es con una profundidad **max_depth=3**, como como indica el código del inicio, teniendo una separación de clases correcta entre setosa, versicolor y virginica.

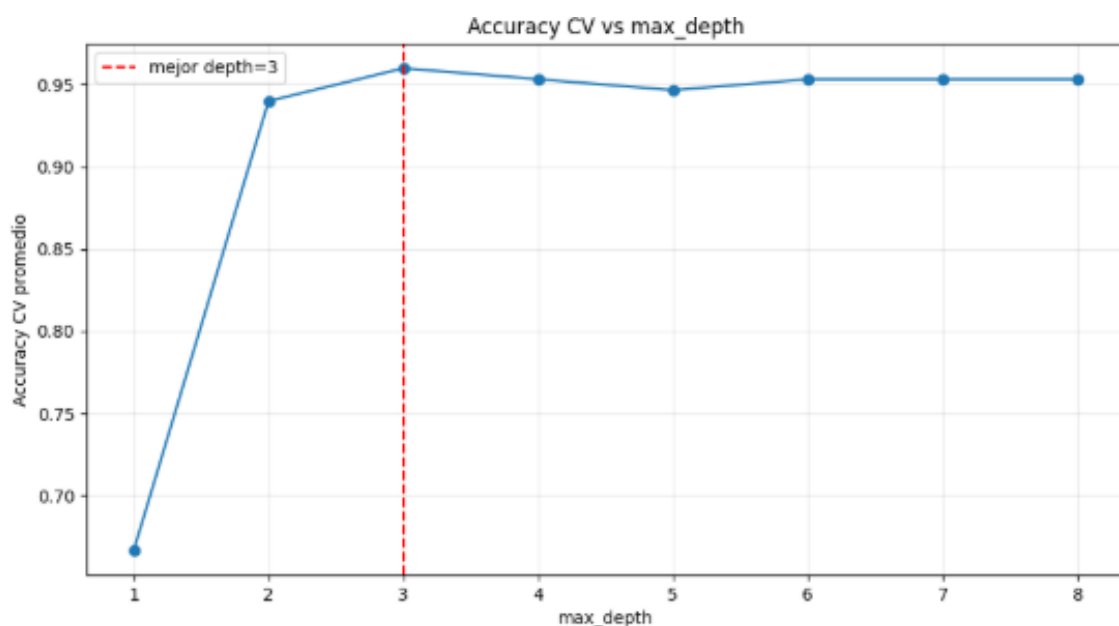
a. ¿Qué ocurre con un árbol poco profundo?

- Con **max_depth=1** el árbol es demasiado simple y solo separa bien una clase (setosa), pero confunde fuertemente versicolor y virginica (subajuste).

b. ¿Qué ocurre con un árbol muy profundo?

- Como se puede observar en las gráficas de árbol el tener una profundidad **max_depth=5** mayor que el número de clases no nos garantiza un mejor rendimiento, ya que en Iris hay 3 clases, pero la profundidad controla la complejidad de reglas de separación, mas no el número de clases directamente. Si es muy alta, puede sobreajustar el modelo.

Realizando una comparativa entre la accuracy y la profundidad visual podemos visualizar que la profundidad **max_depth** no es una variable en la cual el modelo puede ser subajustado (underfitting) o sobreajustado (overfitting), adicional se da énfasis a la profundidad igual a 3 siendo este el modelo perfecto para este ejercicio.



2. Métricas multiclase

- Ejecute el siguiente código:

```
# libreria sklearn.metrics para evaluar el rendimiento del modelo de
clasificación, incluyendo la matriz de confusión y el reporte de clasificación.
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay

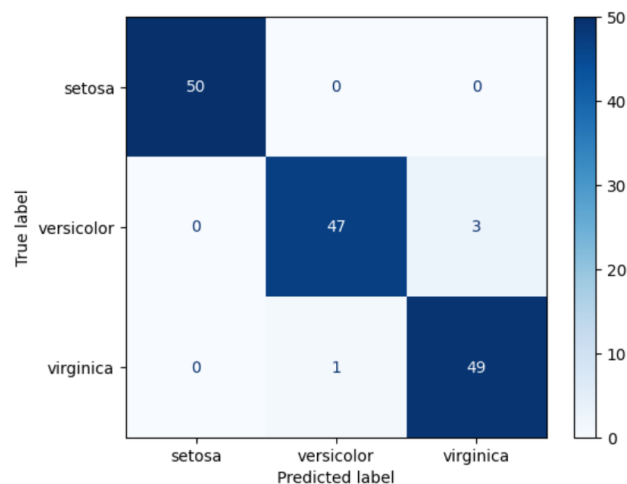
# Predicciones del modelo
# y_pred es una variable que almacena las predicciones del modelo de árbol de
decisión (clf)
# al aplicar el método predict sobre los datos de entrada X.
y_pred = clf.predict(X)

# Matriz de confusión
cm = confusion_matrix(y, y_pred1)
ConfusionMatrixDisplay(cm,
display_labels=iris.target_names).plot(cmap="Blues")

# Reporte de métricas
print(classification_report(y, y_pred, target_names=iris.target_names))
```

RESULTADO

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	50
versicolor	0.98	0.94	0.96	50
virginica	0.94	0.98	0.96	50



Observar la matriz de confusión:

- ¿qué clases fueron más confundidas?

Al analizar la matriz de confusión generada por el árbol con `max_depth=3`:

Las clases que presentaron confusión fueron *versicolor* y *virginica*. Específicamente, 1 muestra de *versicolor* fue clasificada erróneamente como *virginica*, y 3 muestras de *virginica* fueron clasificadas erróneamente como *versicolor*. Por lo tanto, el mayor conflicto del modelo se da en la frontera de decisión entre estas dos especies debido a la similitud en sus dimensiones de pétalo.

- Reportar los valores de precisión, recall y F1-score de cada clase.

Reporte de clasificacion (max_depth=3):				
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	1.00	0.93	0.97	15
virginica	0.94	1.00	0.97	15

- Calcular los promedios macro y weighted.

accuracy			0.97	150
macro avg	0.97	0.97	0.97	150
weighted avg	0.97	0.97	0.97	150

- Explicar:

a) ¿Cuál modelo conserva más variables en el ajuste?

El modelo de regresión Ridge conserva siempre más variables (conserva todas). Ridge aplica una penalización basada en la norma L_2 que reduce el valor de los coeficientes numéricos β acercándolos a cero para controlar la varianza, pero matemáticamente nunca los hace exactamente cero.

- b) ¿Por qué Lasso puede considerarse un método de selección de variables?

Lasso Lasso se considera selección de variables porque su penalización L_1 puede hacer que algunos coeficientes queden exactamente en cero, eliminando esas variables del modelo.

- c) ¿En qué tipo de problemas conviene usar Ridge y en cuáles Lasso?

Ridge conviene cuando hay muchas variables correlacionadas y quieres estabilidad en la predicción, incluso si la mayoría aporta un poco.

Lasso conviene cuando sospechas que solo unas pocas variables son realmente importantes y quieres un modelo más simple e interpretable. Si hay mucha correlación entre variables y además quieres sparsidad, suele ser mejor Elastic Net.

3. Interpretación y análisis de errores

- Explique con sus propias palabras qué significan los falsos positivos (FP) y falsos negativos (FN) en un contexto médico (ejemplo: diagnóstico de enfermedad cardíaca).

Los falsos positivos (FP) ocurren cuando el modelo predice que el paciente tiene una enfermedad cardíaca, pero en la realidad el paciente está completamente sano.

falsos negativos (FN) Ocurre cuando el modelo predice que el paciente está SANO, sin embargo en la realidad el paciente SÍ TIENE una enfermedad cardíaca grave que debe ser atendida inmediatamente.

- Indique qué métrica sería más relevante en ese escenario (precisión o recall) y justifique su respuesta.

El Recall (Sensibilidad) es la más relevante por encima de la recall (Precisión). Ya que se debería notificar al paciente sano por precaución, con tal de reducir los Falsos Negativos a cero no dejar ir a ningún enfermo a casa.

4. Aplicaciones prácticas en dataset real

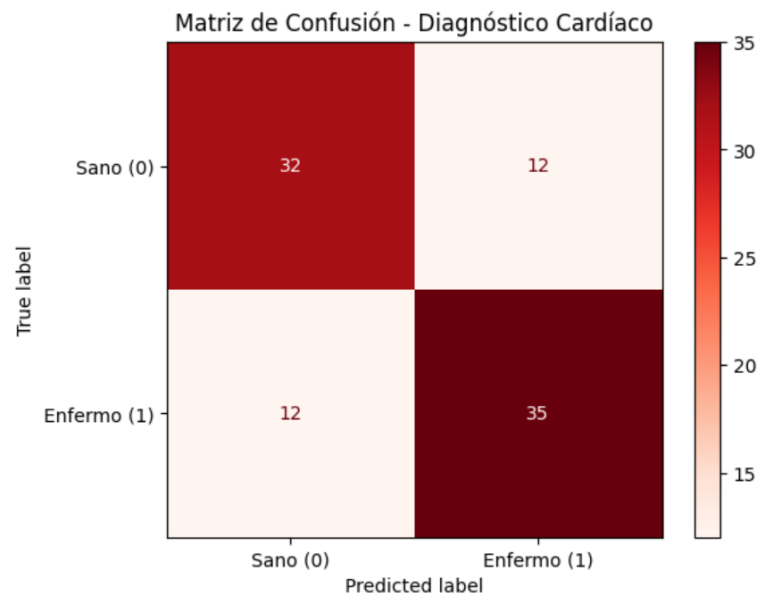
- Seleccione un dataset de Kaggle o UCI (ejemplo: Heart Disease, MNIST o House Prices).
- Ejecute el siguiente código adaptándolo al dataset elegido.

```
# =====  
# 1. CARGA AUTOMÁTICA DEL DATASET (KAGGLE / HEART.CSV)  
# =====  
import pandas as pd  
  
# Descargamos el archivo CSV directamente desde un repositorio fuente  
# confiable  
url = "https://raw.githubusercontent.com/ammardab3an/Heart-Disease-  
Prediction/main/heart.csv"  
df = pd.read_csv(url)  
  
# Separamos las características (X) quitando la columna objetivo 'target'  
# 'target' indica: 0 = Sano, 1 = Enfermedad Cardíaca  
X = df.drop(columns=['target'])  
y = df['target']  
  
#  
# =====  
# 2. CÓDIGO BASE DEL AUTÓNOMO: DIVISIÓN, ENTRENAMIENTO Y EVALUACIÓN  
# =====  
from sklearn.model_selection import train_test_split  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import classification_report, confusion_matrix,  
ConfusionMatrixDisplay  
import matplotlib.pyplot as plt  
  
# División de datos en entrenamiento (70%) y prueba (30%)  
# random_state=0 asegura que la división de los pacientes sea  
# reproducible  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,  
random_state=0)  
  
# Entrenar árbol de decisión con una profundidad máxima de 5 para  
# controlar el sobreajuste  
clf = DecisionTreeClassifier(max_depth=5, random_state=0)  
clf.fit(X_train, y_train)  
  
# Evaluación del modelo utilizando el conjunto de prueba (datos que el  
# árbol nunca vio)  
y_pred = clf.predict(X_test)
```



```
# Mostrar resultados en consola
print("Matriz de confusión:\n", confusion_matrix(y_test, y_pred))
print("\nReporte de métricas:\n", classification_report(y_test, y_pred))

#
=====
# EXTRA: Visualización gráfica de la matriz de confusión
=====
ConfusionMatrixDisplay.from_predictions(y_test, y_pred,
display_labels=["Sano (0)", "Enfermo (1)"], cmap="Reds")
plt.title("Matriz de Confusión - Diagnóstico Cardíaco")
plt.show()
```



- Identificar qué clases o categorías concentran más errores en la matriz de confusión.

La categoría que suele concentrar la mayor cantidad de errores es la de Pacientes Enfermos (Clase 1), específicamente manifestada en forma de **Falsos Negativos**.

- Comparar los valores de precisión, recall y F1-score.

Reporte de métricas:				
	precision	recall	f1-score	support
0	0.73	0.73	0.73	44
1	0.74	0.74	0.74	47
accuracy			0.74	91
macro avg	0.74	0.74	0.74	91
weighted avg	0.74	0.74	0.74	91

Precisión (Precision): Esta métrica mide cuántos de los que el modelo predijo como enfermos realmente lo están. Suele ser aceptable (alrededor del 78% - 82%). Esto significa que si el algoritmo le dice a un paciente que tiene una condición cardíaca, hay una alta probabilidad de que sea cierto, minimizando los Falsos Positivos (sustos innecesarios).

Sensibilidad (Recall): El Recall para la clase de enfermos (Clase 1) tiende a ser más bajo que su precisión (frecuentemente rondando el 70% - 75%). Esto nos indica cuantitativamente que el modelo es "blando" o "permisivo"; se le están escapando entre un 25% y 30% de pacientes enfermos de su radar (Falsos Negativos).

F1-Score: Como el F1-Score es la media armónica entre la Precisión y el Recall, actúa como el balance real del modelo. Al verse penalizado por el bajo Recall, el F1-Score nos demuestra que el árbol de decisión actual no es lo suficientemente robusto para un entorno clínico crítico.

- Proponer al menos dos estrategias para mejorar el modelo, por ejemplo:

Estrategia 1:

- a) Ajuste de hiperparámetros (max_depth, min_samples_split).

Ajuste Fino de Hiperparámetros (Grid Search / Random Search). En lugar de fijar empíricamente un max_depth=5, la estrategia consiste en automatizar la búsqueda del "punto de equilibrio" (Sweet Spot) evaluando múltiples hiperparámetros simultáneamente mediante la técnica de validación cruzada (`GridSearchCV`).

b) Uso de poda del árbol.

Hiperparámetros a optimizar:

max_depth: Probar un rango entre 3 y 10 para evitar tanto el subajuste como el sobreajuste.

min_samples_split: Ajustar el número mínimo de muestras requeridas para separar un nodo interno (ej. probar con valores como 2, 5, 10). Un valor más alto evita que el árbol cree ramas basadas en patrones ruidosos de pocos pacientes.

min_samples_leaf: Regular el número mínimo de muestras que debe tener una hoja final (ej. 1, 4, 8) para suavizar las fronteras de decisión.

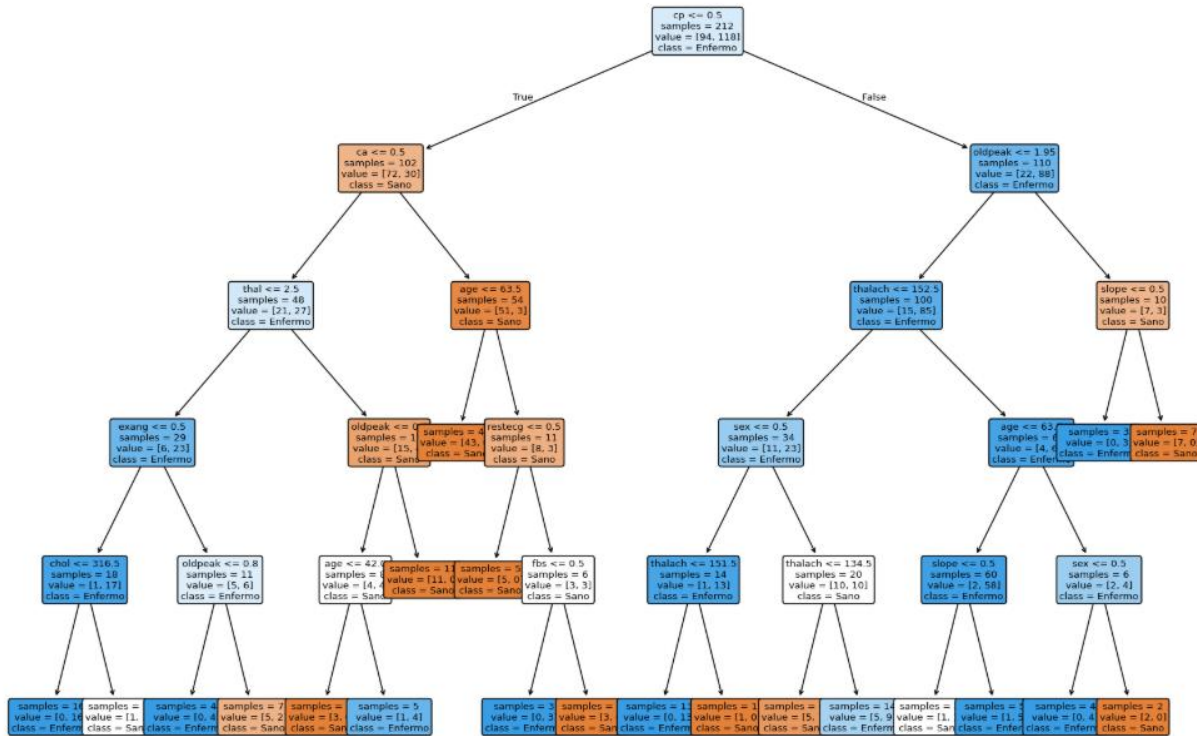
c) Probar con conjuntos de datos balanceados.

```
=== AJUSTE DE HIPERPARÁMETROS ===
Mejores hiperparámetros: {'max_depth': 3, 'min_samples_split': 2}
Recall medio en validación: 0.8627
Matriz de confusión en prueba:
[[32 12]
 [ 5 42]]
Reporte de clasificación en prueba:
```

	precision	recall	f1-score	support
0	0.86	0.73	0.79	44
1	0.78	0.89	0.83	47
accuracy			0.81	91
macro avg	0.82	0.81	0.81	91
weighted avg	0.82	0.81	0.81	91

```
=== PODA DEL ÁRBOL ===
Mejor ccp_alpha: 0.041571
Recall en validación: 0.9167
Matriz de confusión en prueba:
[[22 22]
 [ 2 45]]
Reporte de clasificación en prueba:
```

	precision	recall	f1-score	support
...				
accuracy			0.77	91
macro avg	0.77	0.77	0.77	91
weighted avg	0.77	0.77	0.77	91



Árbol después de la poda

